Digital Signal Processing – HK261 – CO2036

DIGITAL SIGNAL PROCESSING LAB

LAB 4 – Z-TRANSFORM AND CONVOLUTION

Student: Nguyễn Văn Hoàng Phát – 2352896

Overview

This report studies the Z-transform and region of convergence (ROC) of several discrete-time signals and investigates practical signal-processing operations in Scilab. The experimental part includes audio analysis, image processing using the IPCV toolbox, linear convolution using both folding-and-shifting and matrix methods, and circular convolution using the same two approaches. Numerical results, graphical outputs, and energy calculations are used to verify the implementations.

Contents

1 Introduction **2**

2 Objectives **2**

3 Methodology **2**
3.1 Software Environment 2
3.2 Z-transform Definition 2

4 Experimental Content **3**
4.1 Exercise 1 – Z-transform of a Finite-Duration Signal 3
4.2 Exercise 2 – Sum of Two Right-Sided Exponentials 3
4.3 Exercise 3 – Two-Sided Exponential Signal 3
4.4 Exercise 4 – Audio Processing in Scilab 4
4.5 Exercise 5 – Image Processing with IPCV 6
4.6 Exercise 6 – Linear Convolution 7
4.7 Exercise 7 – Circular Convolution 12

5 Discussion **17**

6 References **17**

1 Introduction

The Z-transform is an important tool for analyzing discrete-time signals and systems because it converts a time-domain sequence into a complex frequency-domain representation. In addition to the algebraic expression $X(z)$, the region of convergence is essential because it determines the set of values of z for which the transform exists and helps distinguish between right-sided and left-sided sequences.

Convolution is another fundamental DSP operation. Linear convolution models the output of an LTI system, while circular convolution is closely related to periodic sequences and discrete Fourier transform processing. This laboratory combines these theoretical concepts with practical experiments in Scilab, including audio and image data processing.

2 Objectives

The objectives of this laboratory are:

- to analyze discrete-time signals using the Z-transform;
- to determine the region of convergence for different signal types;
- to investigate Scilab functions for audio processing;
- to investigate image processing using the IPCV toolbox;
- to derive and implement linear convolution using folding-and-shifting and matrix methods;
- to derive and implement circular convolution using folding-and-shifting and matrix methods;
- to compute and visualize the energy of the input and output sequences.

3 Methodology

3.1 Software Environment

The practical exercises were implemented in Scilab 2026.1.0. Built-in sound functions were used for audio processing. For image processing, the IPCV toolbox was installed and loaded through the ATOMS package manager. The convolution algorithms were implemented directly using loops, matrix operations, and discrete-time plots.

3.2 Z-transform Definition

For a discrete-time signal $x(n)$, the bilateral Z-transform is

$$X(z) = \sum_{n=-\infty}^{\infty} x(n) z^{-n}.$$

The ROC contains all values of z for which this sum converges. For a right-sided exponential sequence,

$$\mathcal{Z}\{a^n u(n)\} = \frac{1}{1 - az^{-1}}, \qquad |z| > |a|.$$

For a left-sided exponential sequence,

$$\mathcal{Z}\{a^n u(-n-1)\} = -\frac{1}{1-az^{-1}}, \qquad |z| < |a|.$$

4 Experimental Content

4.1 Exercise 1 – Z-transform of a Finite-Duration Signal

The signal is

$$x(n) = 2\delta(n+2) - \delta(n+1) + 2\delta(n) - 3\delta(n-1) + 4\delta(n-2).$$

Using the shifting property of the impulse,

$$\mathcal{Z}\{\delta(n-k)\} = z^{-k},$$

we obtain

$$\boxed{X(z) = 2z^2 - z + 2 - 3z^{-1} + 4z^{-2}}.$$

The sequence is finite-duration and contains nonzero samples for both negative and positive values of n. Therefore, both $z = 0$ and $z = \infty$ are excluded from the ROC:

$$\boxed{0 < |z| < \infty}.$$

4.2 Exercise 2 – Sum of Two Right-Sided Exponentials

The signal is

$$x(n) = (0.5)^n u(n) + (0.4)^n u(n).$$

The Z-transform is

$$X(z) = \frac{1}{1-0.5z^{-1}} + \frac{1}{1-0.4z^{-1}}.$$

Combining the two terms gives

$$\boxed{X(z) = \frac{2-0.9z^{-1}}{1-0.9z^{-1}+0.2z^{-2}}}.$$

The individual ROCs are $|z| > 0.5$ and $|z| > 0.4$. Their intersection is

$$\boxed{|z| > 0.5}.$$

4.3 Exercise 3 – Two-Sided Exponential Signal

The signal is

$$x(n) = (0.5)^n u(n) + (0.9)^n u(-n-1).$$

For the right-sided component,

$$X_1(z) = \frac{1}{1-0.5z^{-1}}, \qquad |z| > 0.5.$$

For the left-sided component,

$$X_2(z) = -\frac{1}{1 - 0.9z^{-1}}, \qquad |z| < 0.9.$$

Hence,

$$\boxed{X(z) = \frac{1}{1 - 0.5z^{-1}} - \frac{1}{1 - 0.9z^{-1}}}$$

and the overall ROC is the intersection

$$\boxed{0.5 < |z| < 0.9}.$$

This annular ROC is characteristic of a two-sided sequence.

4.4 Exercise 4 – Audio Processing in Scilab

Scilab provides built-in functions for reading, playing, and analyzing audio signals. In this experiment, the built-in `chimes.wav` file was read using `wavread`. The first channel was selected for analysis, the time-domain waveform was plotted, and the FFT was used to obtain a magnitude spectrum.

The experiment reported:

- sampling frequency $F_s = 22050$ Hz;
- bit depth = 16 bits;
- number of channels = 2;
- samples per channel = 13921.

The waveform shows a short audio signal with several transient regions whose amplitudes decay with time. The magnitude spectrum contains several dominant peaks in the low-frequency range, which correspond to the main frequency components of the audio signal.

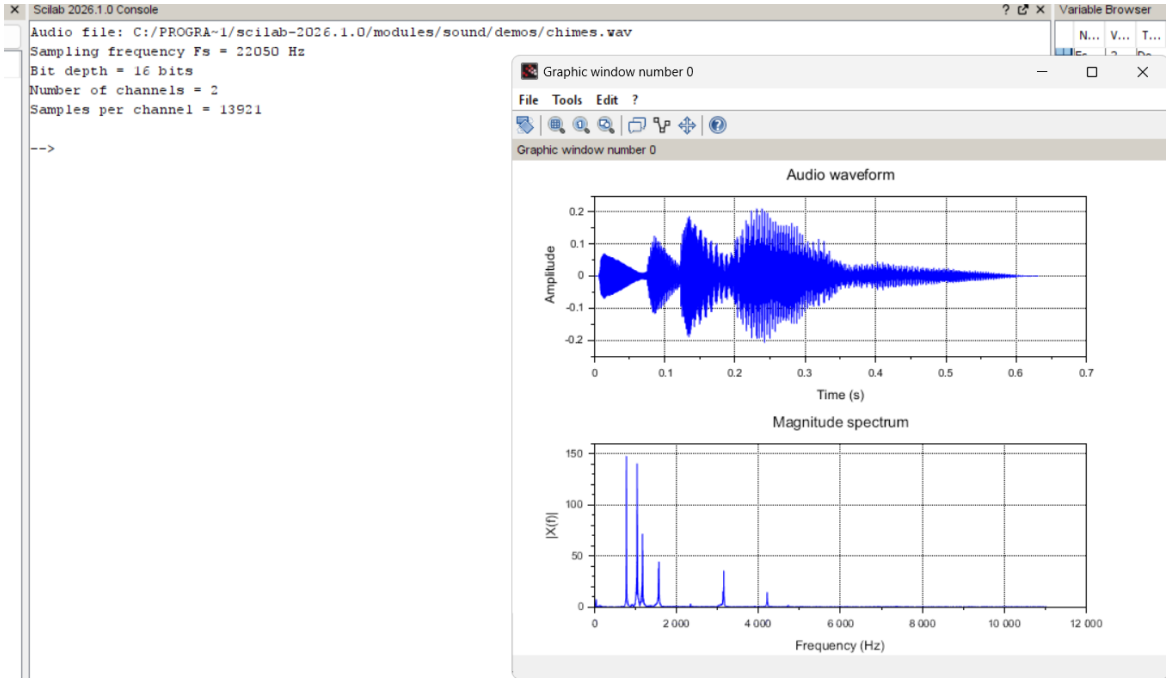


Figure 1: Audio waveform, magnitude spectrum, and audio-file information obtained in Scilab.

Scilab source

```
clear;
clc;

// Exercise 4 - Audio processing demo using built-in Scilab sound
   functions.
// No extra toolbox is required.
//
// This demo:
//   1) reads a WAV file,
//   2) displays the waveform,
//   3) displays a magnitude spectrum,
//   4) optionally plays the sound.

audioFile = SCI + "/modules/sound/demos/chimes.wav";

[y, Fs, bits] = wavread(audioFile);

mprintf("Audio file: %s\n", audioFile);
mprintf("Sampling frequency Fs = %d Hz\n", Fs);
mprintf("Bit depth = %d bits\n", bits);
mprintf("Number of channels = %d\n", size(y, 1));
mprintf("Samples per channel = %d\n", size(y, 2));

// Use the first channel if the audio is multichannel.
x = y(1, :);
N = length(x);
t = (0:N-1) / Fs;

// Frequency-domain representation
X = fft(x);
N2 = floor(N/2);
f = (0:N2-1) * Fs / N;
magX = abs(X(1:N2));

scf();
clf();

subplot(2,1,1);
plot(t, x);
xgrid();
xtitle("Audio waveform", "Time (s)", "Amplitude");

subplot(2,1,2);
plot(f, magX);
xgrid();
xtitle("Magnitude spectrum", "Frequency (Hz)", "|X(f)|");

// Uncomment if you want to hear the audio:
// playsnd(x, Fs);
```

Listing 1: Audio-processing script for Exercise 4.

4.5 Exercise 5 – Image Processing with IPCV

The IPCV toolbox was used to demonstrate basic digital image processing. After the toolbox was loaded, the sample image `baboon.png` was read using `imread`. The original RGB image has size $512 \times 512 \times 3$. It was converted to a grayscale image of size 512×512 using `rgb2gray`. A histogram of grayscale intensities was then generated.

The experiment demonstrates three common image-processing operations: reading an image, converting between color representations, and examining the intensity distribution with a histogram. The histogram shows that the image uses a broad range of gray levels, with relatively large populations in the middle and upper intensity ranges.

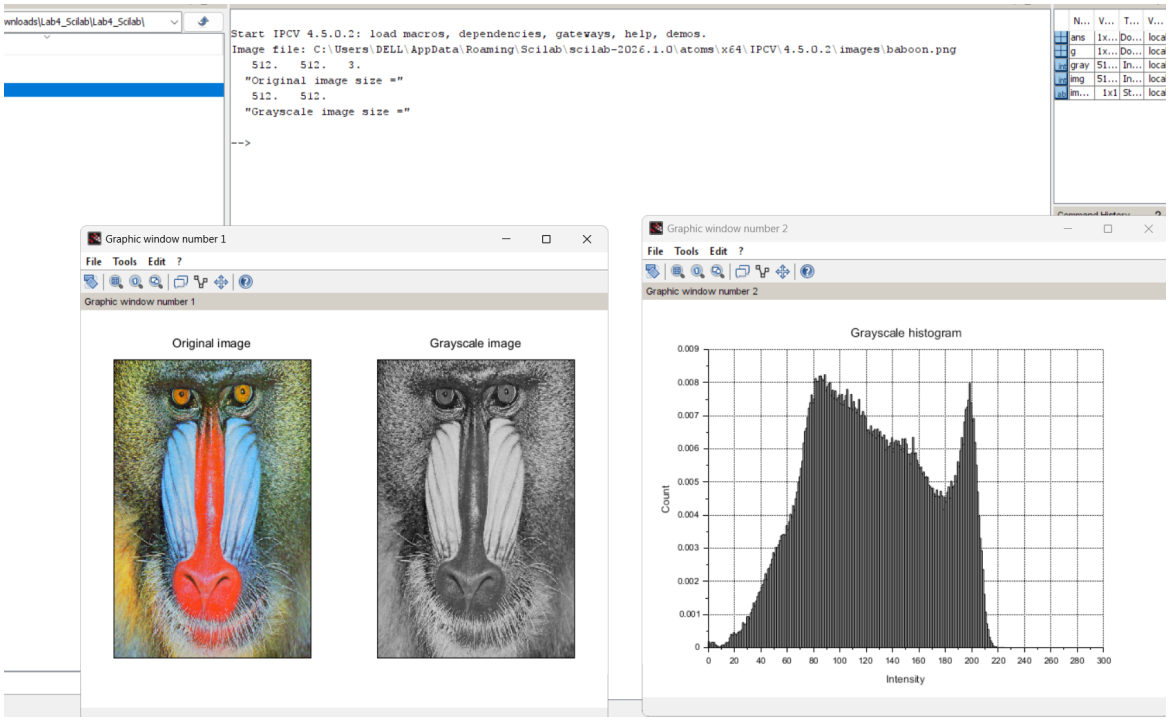


Figure 2: IPCV demonstration: original image, grayscale conversion, and grayscale histogram.

Scilab source

```
clear;
clc;

// Exercise 5 - Image processing demo using IPCV.
//
// First-time installation in the Scilab console:
//   atomsInstall("IPCV")
//
// Then restart Scilab if requested.
// Load the toolbox:
```

```
//   atomsLoad("IPCV")
//
// The demo reads an IPCV sample image, converts it to grayscale,
// displays both images, and plots a grayscale histogram.

atomsLoad("IPCV");

// IPCV includes sample images.
imgPath = fullpath(getIPCVpath() + "/images/baboon.png");
img = imread(imgPath);
gray = rgb2gray(img);

mprintf("Image file: %s\n", imgPath);
disp(size(img), "Original image size =");
disp(size(gray), "Grayscale image size =");

// Display original and grayscale images.
scf();
clf();

subplot(1,2,1);
imshow(img);
xtitle("Original image");

subplot(1,2,2);
imshow(gray);
xtitle("Grayscale image");

// Histogram of grayscale intensity.
// Convert the matrix to a vector for histplot.
g = double(matrix(gray, 1, -1));

scf();
clf();
histplot(0:255, g);
xgrid();
xtitle("Grayscale histogram", "Intensity", "Count");
```

Listing 2: Image-processing script for Exercise 5.

4.6 Exercise 6 – Linear Convolution

Let

$$x(n) = [1, 2, -3, 2, 1], \qquad h(n) = [1, 0, -1],$$

with the first element of each sequence located at $n = 0$.

4.6.1 Convergence of the Convolution Sum

The linear convolution is

$$y(n) = \sum_{k=-\infty}^{\infty} h(k)x(n-k).$$

If both $h(n)$ and $x(n)$ are causal, then $h(k) = 0$ for $k < 0$, and $x(n-k) = 0$ when $k > n$. Hence, for $n \geq 0$,

$$\boxed{y(n) = \sum_{k=0}^{n} h(k)x(n-k)},$$

while $y(n) = 0$ for $n < 0$. For every fixed n, only finitely many terms are nonzero.

If the system has a finite impulse response satisfying

$$h(n) = 0, \qquad n < 0 \text{ or } n > M,$$

then only $k = 0, 1, \ldots, M$ contribute to the convolution. Therefore,

$$\boxed{y(n) = \sum_{k=0}^{M} h(k)x(n-k)}.$$

Again, the convolution contains only $M + 1$ terms for each n.

4.6.2 Folding-and-Shifting Method

For the given impulse response,

$$h(0) = 1, \qquad h(1) = 0, \qquad h(2) = -1.$$

Therefore,

$$y(n) = x(n) - x(n-2).$$

Evaluating the sum for $n = 0, \ldots, 6$ gives

$$\boxed{y(n) = [1, 2, -4, 0, 4, -2, -1]}.$$

The signal energies are

$$E_x = \sum_n |x(n)|^2 = 1^2 + 2^2 + (-3)^2 + 2^2 + 1^2 = \boxed{19},$$

$$E_y = \sum_n |y(n)|^2 = \boxed{42}.$$

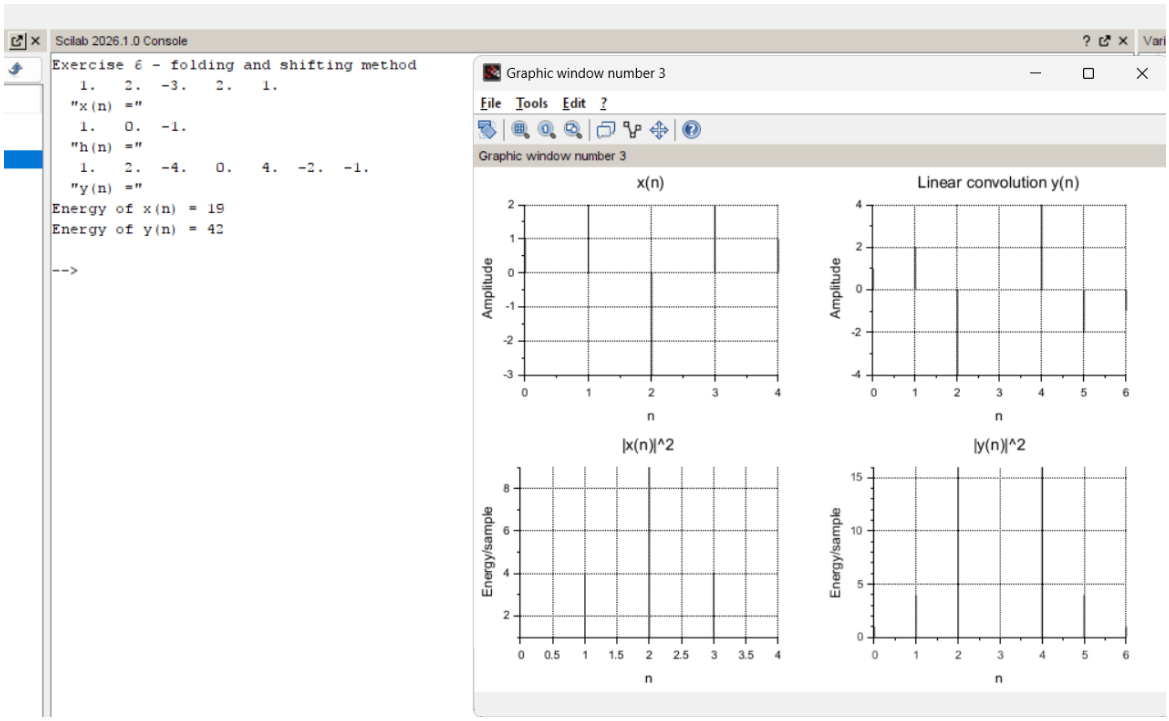


Figure 3: Linear convolution using the folding-and-shifting implementation. The figure also shows the energy per sample.

Scilab source – folding and shifting

```
clear;
clc;

// Exercise 6 - Linear convolution using folding and shifting / direct sum
// x(n) = [1, 2, -3, 2, 1], n starts at 0
// h(n) = [1, 0, -1],       n starts at 0

x = [1 2 -3 2 1];
h = [1 0 -1];

Lx = length(x);
Lh = length(h);
Ly = Lx + Lh - 1;

y = zeros(1, Ly);

// y(n) = sum_k h(k) x(n-k)
for n = 0:Ly-1
    acc = 0;

    for k = 0:Lh-1
        index_x = n - k;

        if index_x >= 0 & index_x <= Lx-1 then
```

```
25              acc = acc + h(k+1) * x(index_x+1);
26          end
27      end
28
29      y(n+1) = acc;
30  end
31
32  mprintf("Exercise 6 - folding and shifting method\n");
33  disp(x, "x(n) =");
34  disp(h, "h(n) =");
35  disp(y, "y(n) =");
36
37  // Energy
38  Ex_seq = abs(x).^2;
39  Ey_seq = abs(y).^2;
40  Ex = sum(Ex_seq);
41  Ey = sum(Ey_seq);
42
43  mprintf("Energy of x(n) = %g\n", Ex);
44  mprintf("Energy of y(n) = %g\n", Ey);
45
46  // Draw x(n), y(n), and energy in the same figure.
47  nx = 0:Lx-1;
48  ny = 0:Ly-1;
49
50  scf();
51  clf();
52
53  subplot(2,2,1);
54  plot2d3(nx, x);
55  xgrid();
56  xtitle("x(n)", "n", "Amplitude");
57
58  subplot(2,2,2);
59  plot2d3(ny, y);
60  xgrid();
61  xtitle("Linear convolution y(n)", "n", "Amplitude");
62
63  subplot(2,2,3);
64  plot2d3(nx, Ex_seq);
65  xgrid();
66  xtitle("|x(n)|^2", "n", "Energy/sample");
67
68  subplot(2,2,4);
69  plot2d3(ny, Ey_seq);
70  xgrid();
71  xtitle("|y(n)|^2", "n", "Energy/sample");
```

Listing 3: Linear convolution using folding and shifting.

4.6.3 Matrix Method

A convolution matrix can be constructed from $x(n)$:

$$\mathbf{X} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -3 & 2 & 1 \\ 2 & -3 & 2 \\ 1 & 2 & -3 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{h} = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}.$$

Then

$$\mathbf{y} = \mathbf{X}\mathbf{h} = \begin{bmatrix} 1 \\ 2 \\ -4 \\ 0 \\ 4 \\ -2 \\ -1 \end{bmatrix}.$$

Thus the matrix method gives the same result as the folding-and-shifting method:

$$\boxed{y(n) = [1, 2, -4, 0, 4, -2, -1]}.$$

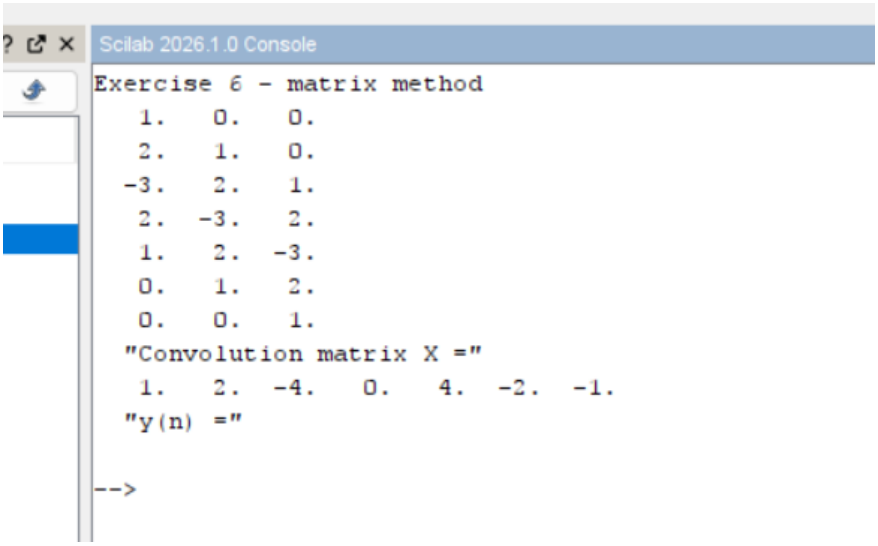


Figure 4: Linear-convolution matrix and output obtained with the matrix method.

Scilab source – matrix method

```
clear;
clc;

// Exercise 6 - Linear convolution using the matrix method

x = [1 2 -3 2 1];
```

```
h = [1 0 -1];

Lx = length(x);
Lh = length(h);
Ly = Lx + Lh - 1;

// Build the linear-convolution matrix from x.
// Xmat * h' gives the convolution result.
Xmat = zeros(Ly, Lh);

for row = 1:Ly
    for col = 1:Lh
        ix = row - col + 1;

        if ix >= 1 & ix <= Lx then
            Xmat(row, col) = x(ix);
        end
    end
end

y_col = Xmat * h';
y = y_col';

mprintf("Exercise 6 - matrix method\n");
disp(Xmat, "Convolution matrix X =");
disp(y, "y(n) =");

// Expected:
// y(n) = [1 2 -4 0 4 -2 -1]
```

Listing 4: Linear convolution using the matrix method.

4.7 Exercise 7 – Circular Convolution

For a periodic sequence $x(n)$ with period N,

$$x(n) = x(n + qN), \qquad q \in \mathbb{Z}.$$

Therefore, every integer index $n - k$ can be mapped into the interval $0, 1, \ldots, N - 1$ using its remainder modulo N:

$$x(n - k) = x[(n - k) \bmod N].$$

If $h(k)$ is nonzero only for $k = 0, 1, \ldots, M - 1$, the convolution over one period is consequently

$$\boxed{y(n) = \sum_{k=0}^{M-1} h(k)x[(n - k) \bmod N]}.$$

This explains the circular-convolution expression used in the exercise.

For

$$x(n) = [1, 2, -3, 2, 1],$$

$$h(n) = [1, 0, -1, -1, 1],$$

and $N = 5$, both sequences have length five.

4.7.1 Folding-and-Shifting Method

Using the modulo index in the circular convolution sum gives

$$\boxed{y(n) = [4, -4, -3, 0, 3]}.$$

The energies are

$$\boxed{E_x = 19}, \qquad \boxed{E_y = 50}.$$

In Scilab, `pmodulo` is used rather than `modulo` so that negative values of $n - k$ are mapped to the nonnegative range $0, 1, \ldots, N - 1$. This avoids invalid array indices.

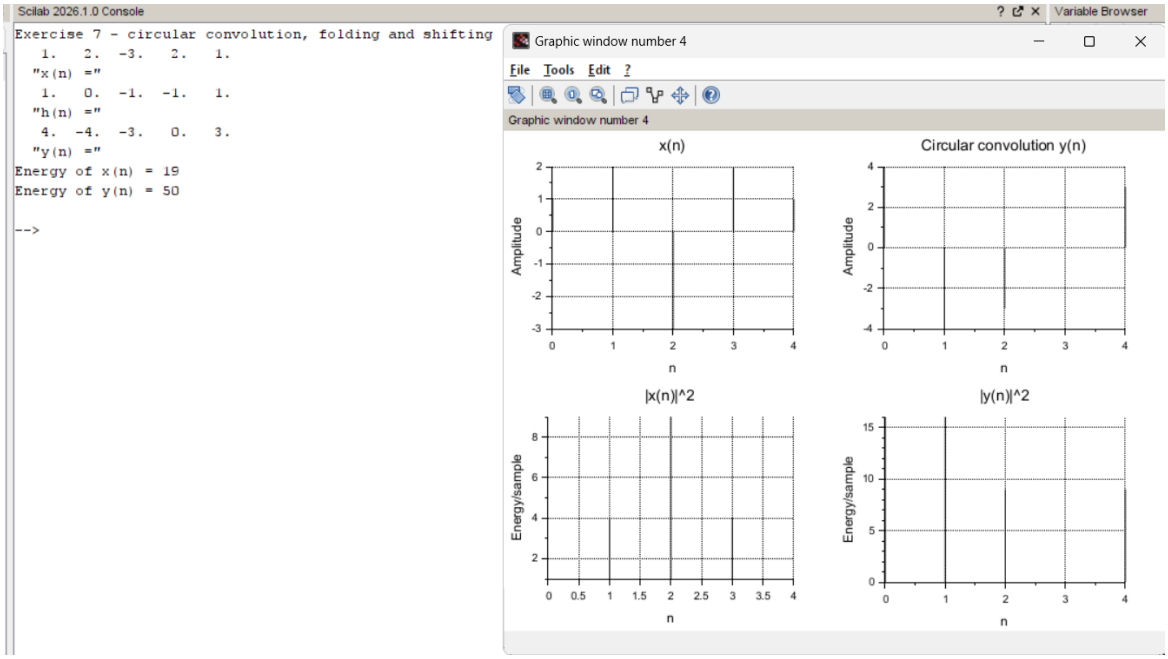


Figure 5: Circular convolution using folding and shifting. The console confirms $y(n) = [4, -4, -3, 0, 3]$, $E_x = 19$, and $E_y = 50$.

Scilab source – folding and shifting

```
clear;
clc;

// Exercise 7 - N-point circular convolution using folding and
   shifting
// x(n) = [1, 2, -3, 2, 1]
// h(n) = [1, 0, -1, -1, 1]
// N = 5

x = [1 2 -3 2 1];
h = [1 0 -1 -1 1];

```

```
12 N = length(x);
13 M = length(h);
14
15 if M <> N then
16     error("For this lab example, x(n) and h(n) must both have
          length N.");
17 end
18
19 y = zeros(1, N);
20
21 // y(n) = sum_{k=0}^{N-1} h(k) x[(n-k) mod N]
22 //
23 // IMPORTANT:
24 // In Scilab, use pmodulo() here instead of modulo().
25 // For negative arguments, pmodulo() always gives an index
26 // in the range 0,...,N-1. Then +1 converts it to Scilab's
27 // 1-based array index.
28
29 for n = 0:N-1
30     acc = 0;
31
32     for k = 0:N-1
33         index_x = pmodulo(n-k, N);   // 0,...,N-1
34         acc = acc + h(k+1) * x(index_x+1);
35     end
36
37     y(n+1) = acc;
38 end
39
40 mprintf("Exercise 7 - circular convolution, folding and shifting\n
      ");
41 disp(x, "x(n) =");
42 disp(h, "h(n) =");
43 disp(y, "y(n) =");
44
45 // Energy
46 Ex_seq = abs(x).^2;
47 Ey_seq = abs(y).^2;
48
49 Ex = sum(Ex_seq);
50 Ey = sum(Ey_seq);
51
52 mprintf("Energy of x(n) = %g\n", Ex);
53 mprintf("Energy of y(n) = %g\n", Ey);
54
55 n = 0:N-1;
56
57 scf();
58 clf();
59
60 subplot(2,2,1);
```

```
plot2d3(n, x);
xgrid();
xtitle("x(n)", "n", "Amplitude");

subplot(2,2,2);
plot2d3(n, y);
xgrid();
xtitle("Circular convolution y(n)", "n", "Amplitude");

subplot(2,2,3);
plot2d3(n, Ex_seq);
xgrid();
xtitle("|x(n)|^2", "n", "Energy/sample");

subplot(2,2,4);
plot2d3(n, Ey_seq);
xgrid();
xtitle("|y(n)|^2", "n", "Energy/sample");
```

Listing 5: Circular convolution using folding and shifting.

4.7.2 Matrix Method

The circular-convolution matrix generated from $h(n)$ is

$$\mathbf{H}_c = \begin{bmatrix} 1 & 1 & -1 & -1 & 0 \\ 0 & 1 & 1 & -1 & -1 \\ -1 & 0 & 1 & 1 & -1 \\ -1 & -1 & 0 & 1 & 1 \\ 1 & -1 & -1 & 0 & 1 \end{bmatrix}.$$

With

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ -3 \\ 2 \\ 1 \end{bmatrix},$$

we obtain

$$\mathbf{y} = \mathbf{H}_c\mathbf{x} = \begin{bmatrix} 4 \\ -4 \\ -3 \\ 0 \\ 3 \end{bmatrix}.$$

Thus, the matrix method verifies the result of the folding-and-shifting method exactly.

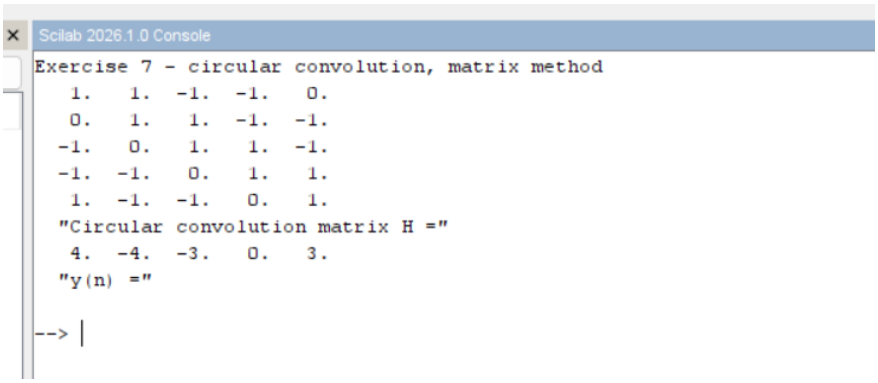


Figure 6: Circular-convolution matrix and output obtained with the matrix method.

Scilab source – matrix method

```
clear;
clc;

// Exercise 7 - Circular convolution using the circulant matrix
   method

x = [1 2 -3 2 1];
h = [1 0 -1 -1 1];

N = length(x);

if length(h) <> N then
    error("For this lab example, x(n) and h(n) must have the same
       length.");
end

// Build Hcirc so that:
// y(n) = sum_m Hcirc(n,m) x(m)
// with Hcirc(n,m) = h[(n-m) mod N]
Hcirc = zeros(N, N);

for n = 0:N-1
    for m = 0:N-1
        index_h = pmodulo(n-m, N);   // 0,...,N-1
        Hcirc(n+1, m+1) = h(index_h+1);
    end
end

y_col = Hcirc * x';
y = y_col';

mprintf("Exercise 7 - circular convolution, matrix method\n");
disp(Hcirc, "Circular convolution matrix H =");
disp(y, "y(n) =");

// Expected:
```

```
35 // y(n) = [4 -4 -3 0 3]
```

Listing 6: Circular convolution using the matrix method.

5 Discussion

The theoretical Z-transform exercises show that the ROC is an essential part of the transform description. Exercise 2 is purely right-sided and therefore has an ROC outside its outermost pole, whereas Exercise 3 is two-sided and has an annular ROC between the two pole magnitudes.

The audio and image experiments demonstrate that Scilab can process real digital data in addition to numerical sequences. The audio example combines time-domain inspection with frequency-domain analysis through the FFT, while the image example uses IPCV to perform grayscale conversion and histogram analysis.

For Exercise 6, both linear-convolution implementations produced $[1, 2, -4, 0, 4, -2, -1]$, confirming the correctness of the folding-and- shifting and matrix approaches. For Exercise 7, both circular- convolution implementations produced $[4, -4, -3, 0, 3]$. The use of `pmodulo` is important in Scilab because it maps negative circular indices into valid nonnegative remainders before conversion to Scilab's one-based array indexing.

Overall, all required theoretical results and practical Scilab experiments were completed successfully, and the numerical outputs from independent methods agree with each other.

6 References

1. Digital Signal Processing, Lab #4 handout, course material.
2. Scilab Help System, sound-processing functions including `wavread`, `fft`, and `playsnd`.
3. IPCV – Image Processing and Computer Vision Toolbox for Scilab.